Vaishnavi Kouru

Compilers & ML Systems

+1 650 305 5350 | vkouru07@gmail.com | portfolio | linkedin.com/in/vaishnavi-kouru | github | U.S. Citizen

EDUCATION

B.S. Computer Science, San Jose State University **2023 – 2026**

Relevant Coursework: Concepts of Compiler Design (CS 153, in progress), Computer Architecture (CS 147), Computer Systems (CS 47), Operating Systems (CS 149), Data Structures and Algorithms (CS 146), Formal Languages and Computability (CS 154), Programming Paradigms (CS 152, in progress), Individual Studies for Honors (CS 180H, in progress)

Positions: Grader for CS 154 (Formal Languages and Computability); CS Study Lab Tutor

SKILLS

Languages: C/C++, Rust, Python, Java, TypeScript, SQL, Bash

Compilers: LLVM (loop vectorizer, optimization remarks, target features), Clang, MLIR, ANTLR, JVM bytecode (Jasmin), RISC-V Vector Extension (RVV) code generation

Performance Analysis: assembly inspection, dynamic instruction counts under QEMU, Compiler Explorer (Godbolt)

ML Systems: ONNX, INT8 quantization, static-shape graph lowering, edge NPU deployment, NVIDIA Isaac Lab

Tools: Linux (Ubuntu), Unix, macOS, Git, GitHub (issues, pull requests), UART/serial bring-up

RESEARCH & UPSTREAM WORK

Silent Degradation during Auto-Vectorization of bf16/fp16 on RISC-V Under LLVM *Paper [forthcoming]*

- Analyzed LLVM's RISC-V auto-vectorizer on a multiply-accumulate kernel across fp32, int8, bf16, and fp16 using assembly-level inspection, LLVM optimization remarks, and dynamic instruction counts under QEMU user-mode emulation.
- Found that under baseline `-march=rv64gcv`, int8 and fp32 vectorize cleanly while bf16 and fp16 fall back to scalar-heavy code (fp16 using per-element software conversion calls), executing **17.0**× and **75.9**× more dynamic instructions than with `zvfbfmin`/`zvfbfwma` and `zvfhmin` enabled.
- Showed that compiler diagnostics report the degradation but do not distinguish flag-fixable limitations from inherent conversion costs, leaving no direct signal toward the fix.

LLVM Issue: Unclear Loop Vectorizer Remark on RISC-V *llvm/llvm-project #232987*

- Reported that when `half`/`bfloat` conversions block scalable vectorization, the remark names neither the unsupported type nor the extension that would fix it; filed with a minimal C reproducer and Godbolt link.

Undergraduate Researcher, Information Flow Security in Racket Spring 2027 (incoming)

San Jose State University | Advisor: Prof. Thomas Austin

- Will study how faceted values and Racket macros handle implicit information flows through program branches.

COMPILER PROJECTS

Graph-Level and Low-Level Optimization Passes for an MNIST Model *Class Project, Fall 2026 | LLVM/MLIR*

- Building optimization passes at two levels: graph-level transformations such as operator fusion, and low-level code generation such as vectorization, with the goal of beating `torch.compile` on inference performance.

Music Language Compiler *CS 153 Project, Fall 2026 | ANTLR, Jasmin, JVM*

- Designing a domain-specific language for describing and playing music, with variables, math, and loops, compiled through an ANTLR front end to Jasmin assembly for the JVM.

Pascal Interpreter and Pascal-to-Java Compiler *CS 153 Coursework | Java, ANTLR | github*

- Extended an ANTLR grammar and interpreter for a Pascal subset with `if`, `while`, `for`, and `case` statements, verified against expected-output test programs; developed with a team through feature branches and pull requests.
- Added a built-in hashtable type to the grammar and type system, and implemented back-end translation of named and unnamed Pascal records to Java classes.

EXPERIENCE

(Modalix Edge AI Development Kit) Industry Project Apr 2026 – Present

SiMa.ai — San Jose

- Designed "Little ACT," an Action Chunking Transformer policy (TinyCNN encoder, 2+2 transformer layers) restricted to the Modalix-supported op set (Conv/Linear/MatMul/Softmax/LayerNorm/ReLU) with static shapes
- Generated cube-stacking demonstration data in NVIDIA Isaac Lab, trained the policy, and exported it to static-shape ONNX (opset 17); now quantizing to INT8 and compiling for Modalix with the SiMa Palette Model SDK
- Brought up the DevKit from scratch (UART serial bring-up, Linux driver conflict resolution) and documented the full pipeline

Software Developer Apr 2025 – Present

Midtown Family Services — San Jose, CA

- Built a tablet- and mobile-friendly Salesforce registration app and dashboards for a community food pantry.